TP3 : Minage et Proof of Work : sécuriser une blockchain avec Python

Matière : Blockchain

Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- Comprendre le principe du minage dans une blockchain.
- Expliquer le fonctionnement du Proof of Work (preuve de travail).
- Comprendre le rôle du nonce dans le calcul d'un hash.
- Implémenter un mécanisme de minage simple en Python.
- Observer l'influence de la difficulté sur le temps de minage.
- Comprendre comment le Proof of Work contribue à la sécurité d'une blockchain.
- Analyser les avantages et les limites de ce mécanisme.

Partie I — Cours -- Rappel : comprendre le minage et le Proof of Work

1. Rappel : fonctionnement d'une blockchain

Dans le TP précédent, nous avons construit une blockchain simplifiée en Python.

Chaque bloc contenait :

- Un identifiant.
- Une transaction.
- Le hash du bloc précédent.
- Son propre hash.

Nous avons également développé une fonction permettant de vérifier la cohérence de la chaîne.

Prenons l'exemple suivant :

Bloc 1

Transaction : Alice envoie 10 € à Bob
Hash précédent : 0
Hash actuel : A123

Bloc 2

Transaction : Bob envoie 5 € à Charlie
Hash précédent : A123
Hash actuel : B456

Bloc 3

Transaction : Charlie envoie 2 € à Alice
Hash précédent : B456
Hash actuel : C789

Les hash présentés sont fictifs.

Dans notre programme précédent, le hash d'un bloc était calculé immédiatement.

Cependant, cette opération est relativement simple et rapide.

Une personne pourrait donc modifier une transaction et recalculer les hash des blocs suivants.

Comment rendre cette opération plus difficile ?

Une solution consiste à imposer un travail de calcul avant d'accepter un nouveau bloc.

C'est le principe du **Proof of Work**.

2. Qu'est-ce que le minage ?

Le minage est une opération qui consiste à effectuer des calculs pour trouver un hash respectant certaines conditions imposées par le protocole.

Dans une blockchain utilisant le Proof of Work, comme Bitcoin, les mineurs utilisent leur puissance de calcul pour rechercher une preuve de travail valide.

Cette preuve permet de proposer un nouveau bloc au réseau.

Exemple simple

Supposons que nous souhaitions créer un bloc contenant la transaction :

Alice envoie 10 € à Bob.

Le programme calcule le hash de ce bloc.

Mais nous ajoutons une règle :

Le hash du bloc doit commencer par trois zéros.

Exemples fictifs :

Hash 1 : A7B92F... → Refusé

Hash 2 : 91F0AC... → Refusé

Hash 3 : 0B82D1... → Refusé

Hash 4 : 000A72... → Accepté

Le programme doit donc effectuer plusieurs tentatives jusqu'à trouver un hash respectant la condition.

Cette recherche constitue une version simplifiée du minage.

Dans Bitcoin, la condition réelle consiste à obtenir un hash inférieur à une valeur cible. L'utilisation de zéros au début du hash est une simplification pédagogique.

3. Qu'est-ce que le Proof of Work ?

Proof of Work signifie « preuve de travail ».

Il s'agit d'un mécanisme dans lequel un participant doit démontrer qu'il a effectué un certain travail de calcul.

Le principe repose sur deux idées :

Première idée : trouver une solution doit demander du travail.

Le mineur doit effectuer plusieurs calculs avant de trouver un hash valide.

Deuxième idée : vérifier la solution doit être facile.

Une fois le hash trouvé, les autres participants peuvent rapidement vérifier qu'il respecte la condition imposée.

Exemple

Nous imposons la règle suivante :

Le hash doit commencer par `000`.

Un mineur essaie différentes valeurs jusqu'à trouver une solution.

Lorsqu'il trouve un hash valide, il communique le bloc et la valeur utilisée.

Les autres participants recalculent le hash et vérifient la condition.

Il n'est donc pas nécessaire de refaire toutes les tentatives du mineur pour vérifier son travail.

Pourquoi utiliser ce mécanisme ?

Le Proof of Work rend la création de blocs coûteuse en ressources de calcul.

Si une personne souhaite modifier un ancien bloc, elle devra recalculer sa preuve de travail ainsi que celles des blocs suivants, puis rivaliser avec le travail de la chaîne reconnue par le réseau.

Dans une véritable blockchain, cette protection dépend également des règles de consensus et de la puissance de calcul du réseau.

4. Le rôle du nonce

Pour rechercher un hash valide, nous devons pouvoir modifier une valeur dans le bloc sans changer la transaction.

Cette valeur est appelée **nonce**.

Un nonce est une valeur numérique que le programme fait varier pendant la recherche.

Dans notre TP, nous commencerons avec :

nonce = 0

Le programme calculera ensuite le hash du bloc.

Si le hash ne respecte pas la condition, le nonce sera augmenté :

nonce = 1

Puis :

nonce = 2

Puis :

nonce = 3

Et ainsi de suite.

Le processus continue jusqu'à trouver un hash valide.

Exemple simplifié

Nonce	Hash fictif	Condition : commence par 000
0	A72B19...	Non
1	8F21C4...	Non
2	02A9D1...	Non
3	000B72...	Oui

Dans cet exemple, le nonce 3 permet de trouver un hash respectant la condition.

Les valeurs sont fictives et servent uniquement à illustrer le mécanisme.

À retenir

Le nonce ne représente pas une transaction ou un montant.

Il s'agit d'une valeur utilisée pour produire différentes empreintes jusqu'à trouver une solution valide.

5. Comprendre la difficulté du minage

La difficulté détermine à quel point il est compliqué de trouver un hash valide.

Dans notre programme, nous représenterons la difficulté par le nombre de zéros exigés au début du hash.

Par exemple :

Difficulté 1

Le hash doit commencer par :

```
0
```

Difficulté 2

Le hash doit commencer par :

```
00
```

Difficulté 3

Le hash doit commencer par :

`000`

Difficulté 4

Le hash doit commencer par :

`0000`

Plus le nombre de zéros exigés augmente, plus il devient difficile de trouver un hash valide.

Pourquoi ?

Un caractère hexadécimal peut prendre 16 valeurs différentes :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f.

La probabilité qu'un hash SHA-256 commence par un zéro est d'environ 1 sur 16.

Pour deux zéros, elle est d'environ 1 sur 256.

Pour trois zéros, elle est d'environ 1 sur 4 096.

Ces probabilités permettent d'estimer le nombre de tentatives nécessaires, mais ne garantissent pas le nombre exact de calculs pour un bloc donné.

Difficulté	Condition	Nombre moyen théorique de tentatives
1	Commence par 0	16
2	Commence par 00	256
3	Commence par 000	4 096
4	Commence par 0000	65 536
5	Commence par 00000	1 048 576

Le temps de minage dépend également des performances de l'ordinateur.

6. Mesurer le temps de minage

Pour mesurer le temps nécessaire au minage, nous utiliserons la bibliothèque Python `time`.

Exemple :

```
import time

debut = time.perf_counter()

# Instructions a mesurer

fin = time.perf_counter()

duree = fin - debut

print("Duree :", duree, "secondes")
```

La fonction `time.perf_counter()` fournit un compteur adapté à la mesure de durées.

Nous l'utiliserons pour comparer les temps de minage selon différentes difficultés.

Partie II — TP pratique guidé

Exercice 1 — Rechercher un hash commençant par des zéros

Étape 1 : préparation

Créez un dossier nommé :

`TP3_Blockchain`

Dans ce dossier, créez un fichier :

`exercice1.py`

Étape 2 : écrire le programme

```
import hashlib

message = "Blockchain"
nonce = 0

while True:

    contenu = message + str(nonce)

    hash_resultat = hashlib.sha256(
        contenu.encode("utf-8")
    ).hexdigest()

    if hash_resultat.startswith("000"):
        break

    nonce += 1

print("Message :", message)
print("Nonce :", nonce)
print("Hash :", hash_resultat)
```

Travail demandé

1. Exécutez le programme.
2. Notez la valeur du nonce trouvée.
3. Notez le hash obtenu.
4. Vérifiez que le hash commence par trois zéros.
5. Remplacez le message `Blockchain` par `Bitcoin`.
6. Exécutez à nouveau le programme.
7. Comparez les valeurs obtenues.

Questions

1. Quel est le rôle de la variable `nonce` ?

2. Pourquoi utilisons-nous une boucle `while` ?
3. Quel est le rôle de la méthode `startswith()` ?
4. Pourquoi le programme effectue-t-il plusieurs calculs ?
5. Le même message produit-il toujours le même nonce dans ce programme ? Justifiez.
6. Pourquoi la modification du message peut-elle changer le nonce trouvé ?

Exercice 2 — Modifier la difficulté du minage

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice2.py`

Étape 2 : écrire le programme

```
import hashlib
import time

message = "Blockchain"
difficulte = 3

nonce = 0

debut = time.perf_counter()

while True:

    contenu = message + str(nonce)

    hash_resultat = hashlib.sha256(
        contenu.encode("utf-8")
    ).hexdigest()

    if hash_resultat.startswith("0" * difficulte):
        break

    nonce += 1

fin = time.perf_counter()

duree = fin - debut

print("Difficulte :", difficulte)
print("Nonce :", nonce)
print("Hash :", hash_resultat)
print("Temps de minage :", duree, "secondes")
```

Travail demandé

1. Exécutez le programme avec une difficulté égale à 1.
2. Notez le nombre de tentatives et le temps de minage.

3. Recommencez avec une difficulté égale à 2.
4. Recommencez avec une difficulté égale à 3.
5. Recommencez avec une difficulté égale à 4.
6. Comparez les résultats obtenus.

Dans notre programme, les tentatives commencent avec le nonce 0. Le nombre total de tentatives correspond donc à `nonce + 1`.

Tableau à compléter

Difficulté	Nombre de tentatives	Temps de minage (secondes)
1		
2		
3		
4		

Questions

1. Comment évolue le nombre de tentatives lorsque la difficulté augmente ?
2. Comment évolue le temps de minage ?
3. Le temps de minage augmente-t-il toujours de manière régulière ?
4. Pourquoi deux difficultés successives peuvent-elles produire des temps très différents ?
5. Quel est l'intérêt d'augmenter la difficulté dans un système Proof of Work ?

Exercice 3 — Ajouter le minage à la classe Bloc

Dans le TP2, nous avons créé une classe `Bloc` qui calculait automatiquement son hash.

Nous allons maintenant modifier cette classe pour qu'un bloc doive être miné avant d'être considéré comme valide.

Étape 1 : créer le fichier

Créez un fichier nommé :

`exercice3.py`

Étape 2 : définir la classe Bloc

```
import hashlib
import time

class Bloc:

    def __init__(self, index, transaction, previous_hash):

        self.index = index
        self.transaction = transaction
        self.previous_hash = previous_hash
        self.nonce = 0
```

```
        self.hash = self.calculer_hash()

    def calculer_hash(self):

        contenu = (
            str(self.index)
            + "|"
            + self.transaction
            + "|"
            + self.previous_hash
            + "|"
            + str(self.nonce)
        )

        return hashlib.sha256(
            contenu.encode("utf-8")
        ).hexdigest()

    def miner_bloc(self, difficulte):

        cible = "0" * difficulte

        while not self.hash.startswith(cible):

            self.nonce += 1
            self.hash = self.calculer_hash()

        print("Bloc mine :", self.index)
        print("Nonce :", self.nonce)
        print("Hash :", self.hash)
```

Étape 3 : créer et miner un bloc

Ajoutez le code suivant :

```
bloc1 = Bloc(
    1,
    "Alice envoie 10 euros a Bob",
    "0"
)

bloc1.miner_bloc(3)
```

Travail demandé

1. Exécutez le programme.
2. Observez la valeur du nonce.
3. Observez le hash du bloc.
4. Modifiez la difficulté de 3 à 4.
5. Exécutez à nouveau le programme.
6. Comparez les valeurs obtenues.

Questions

1. Quelle différence existe-t-il entre cette classe et celle du TP2 ?
2. Pourquoi avons-nous ajouté l'attribut `nonce` ?
3. Quel est le rôle de la méthode `miner_bloc()` ?
4. Pourquoi le hash est-il recalculé à chaque modification du nonce ?
5. Pourquoi le minage s'arrête-t-il lorsqu'un hash valide est trouvé ?

Exercice 4 — Miner une blockchain de trois blocs

Étape 1 : reprendre la classe Bloc

Créez un fichier nommé :

`exercice4.py`

Copiez la classe `Bloc` de l'exercice 3.

Étape 2 : créer les blocs

Ajoutez le programme suivant :

```
difficulte = 3

bloc1 = Bloc(
    1,
    "Alice envoie 10 euros a Bob",
    "0"
)

bloc1.miner_bloc(difficulte)

bloc2 = Bloc(
    2,
    "Bob envoie 5 euros a Charlie",
    bloc1.hash
)

bloc2.miner_bloc(difficulte)

bloc3 = Bloc(
    3,
    "Charlie envoie 2 euros a Alice",
    bloc2.hash
)

bloc3.miner_bloc(difficulte)

blockchain = [bloc1, bloc2, bloc3]
```

Étape 3 : afficher la blockchain

Ajoutez :

```
for bloc in blockchain:

    print("--------------------------")
    print("Bloc :", bloc.index)
    print("Transaction :", bloc.transaction)
    print("Hash precedent :", bloc.previous_hash)
    print("Nonce :", bloc.nonce)
    print("Hash :", bloc.hash)
```

Travail demandé

1. Exécutez le programme.
2. Notez le nonce trouvé pour chaque bloc.
3. Comparez les hash des trois blocs.
4. Vérifiez que chaque bloc respecte la difficulté imposée.
5. Vérifiez que le hash précédent du Bloc 2 correspond au hash du Bloc 1.
6. Vérifiez que le hash précédent du Bloc 3 correspond au hash du Bloc 2.

Questions

1. Pourquoi doit-on miner le Bloc 1 avant de créer le Bloc 2 ?
2. Pourquoi le Bloc 2 dépend-il du hash final du Bloc 1 ?
3. Les trois blocs possèdent-ils nécessairement le même nonce ?
4. Pourquoi le minage d'un bloc dépend-il de son contenu ?
5. Que se passerait-il si nous modifiions une transaction dans le Bloc 1 après le minage ?

Partie III — Exercices d'application

Exercice 5 — Vérifier une blockchain avec Proof of Work

Dans le TP2, nous avons développé une fonction permettant de vérifier l'intégrité d'une blockchain.

Nous allons maintenant ajouter une troisième vérification : chaque bloc doit respecter la difficulté imposée.

Étape 1 : compléter le programme

Reprenez le fichier `exercice4.py`.

Ajoutez la fonction suivante :

```
def verifier_blockchain(blockchain, difficulte):

    cible = "0" * difficulte

    for i in range(len(blockchain)):

        bloc = blockchain[i]
```

```
        # Verification du contenu
        if bloc.hash != bloc.calculer_hash():
            return False

        # Verification de la preuve de travail
        if not bloc.hash.startswith(cible):
            return False

        # Verification du chainage
        if i > 0:

            bloc_precedent = blockchain[i - 1]

            if bloc.previous_hash != bloc_precedent.hash:
                return False

    return True
```

Étape 2 : tester la fonction

Ajoutez :

```
print(
    "Blockchain valide :",
    verifier_blockchain(blockchain, difficulte)
)
```

Étape 3 : modifier une transaction

Ajoutez :

```
bloc1.transaction = "Alice envoie 1000 euros a Bob"

print(
    "Blockchain valide apres modification :",
    verifier_blockchain(blockchain, difficulte)
)
```

Travail demandé

1. Exécutez le programme avant toute modification.
2. Notez le résultat de la vérification.
3. Modifiez la transaction du Bloc 1.
4. Exécutez la fonction de vérification.
5. Identifiez la condition qui détecte la modification.
6. Expliquez pourquoi une nouvelle preuve de travail serait nécessaire.

Questions

1. Pourquoi vérifions-nous le hash du bloc ?
2. Pourquoi vérifions-nous le nombre de zéros au début du hash ?
3. Pourquoi vérifions-nous le hash du bloc précédent ?

4. Quelle est la différence entre cette fonction et celle développée dans le TP2 ?
5. Le Proof of Work empêche-t-il totalement la modification d'une blockchain ? Justifiez.

Exercice 6 — Étudier l'influence de la difficulté

Nous allons mesurer le temps nécessaire pour miner un même bloc avec différentes difficultés.

Étape 1 : préparer l'expérience

Reprenez la classe `Bloc` de l'exercice 3.

Créez un fichier nommé :

`exercice6.py`

Ajoutez le code suivant après la définition de la classe :

```
difficultes = [1, 2, 3, 4]

for difficulte in difficultes:

    bloc = Bloc(
        1,
        "Alice envoie 10 euros a Bob",
        "0"
    )

    debut = time.perf_counter()

    bloc.miner_bloc(difficulte)

    fin = time.perf_counter()

    print("Difficulte :", difficulte)
    print("Temps :", fin - debut)
    print("Tentatives :", bloc.nonce + 1)
    print("--------------------------")
```

Travail demandé

1. Exécutez le programme.
2. Relevez les temps obtenus.
3. Relevez le nombre de tentatives.
4. Complétez le tableau suivant.

Difficulté	Nombre de tentatives	Temps (secondes)
1		
2		
3		
4		

Étape 2 : représenter les résultats

À l'aide de Matplotlib, réalisez un graphique représentant :

- Sur l'axe horizontal : la difficulté.
- Sur l'axe vertical : le temps de minage.

Vous pouvez utiliser le modèle suivant :

```
import matplotlib.pyplot as plt

difficultes = [1, 2, 3, 4]

temps = []

# Ajouter ici les temps mesures

plt.plot(difficultes, temps, marker="o")

plt.xlabel("Difficulte")
plt.ylabel("Temps de minage (secondes)")
plt.title("Influence de la difficulte sur le minage")

plt.grid(True)
plt.show()
```

Complétez la liste `temps` avec les valeurs mesurées.

Questions

1. Quelle relation observez-vous entre la difficulté et le temps de minage ?
2. Pourquoi le temps de minage augmente-t-il généralement avec la difficulté ?
3. Le nombre de tentatives est-il proportionnel à la difficulté ?
4. Pourquoi les résultats peuvent-ils varier d'un ordinateur à un autre ?
5. Quel serait l'effet d'une difficulté très élevée sur les performances du système ?

Partie IV — Étude de cas : tentative de modification d'une blockchain

Exercice 7 — Comprendre le coût d'une modification

Considérons une blockchain contenant quatre blocs déjà minés.

Chaque bloc respecte une difficulté de 4.

Un utilisateur malveillant souhaite modifier la transaction du Bloc 2.

Situation initiale

Bloc 1

Transaction : Alice → Bob : 10 €

↓

Bloc 2

Transaction : Bob → Charlie : 5 €

↓

Bloc 3

Transaction : Charlie → David : 3 €

↓

Bloc 4

Transaction : David → Alice : 1 €

Situation modifiée

L'utilisateur remplace la transaction du Bloc 2 par :

Bob → Charlie : 500 €

Travail demandé

1. Identifiez les blocs affectés par cette modification.
2. Expliquez pourquoi le hash du Bloc 2 doit être recalculé.
3. Expliquez pourquoi le Bloc 3 est également concerné.
4. Indiquez les opérations nécessaires pour reconstruire une chaîne cohérente.
5. Expliquez pourquoi le Proof of Work augmente le coût de cette opération.

Questions de réflexion

1. Un utilisateur peut-il recalculer les preuves de travail de plusieurs blocs ?
2. Pourquoi une telle modification devient-elle plus coûteuse lorsque la difficulté augmente ?
3. Dans une véritable blockchain décentralisée, pourquoi ne suffit-il pas de modifier sa propre copie de la chaîne ?
4. Quel rôle jouent les autres participants du réseau ?
5. Pourquoi la sécurité dépend-elle également du mécanisme de consensus ?

Partie V — Questions de compréhension

Répondez individuellement aux questions suivantes.

1. Qu'est-ce que le minage dans une blockchain ?
2. Que signifie Proof of Work ?
3. Quel est le rôle du nonce ?

4. Pourquoi utilise-t-on une fonction de hachage pendant le minage ?
5. Pourquoi le mineur doit-il effectuer plusieurs tentatives ?
6. Quelle est la différence entre calculer un hash et miner un bloc ?
7. Comment représente-t-on la difficulté dans notre programme ?
8. Pourquoi le temps de minage augmente-t-il généralement avec la difficulté ?
9. Pourquoi est-il facile de vérifier une preuve de travail valide ?
10. Pourquoi un bloc doit-il être miné avant de créer le bloc suivant dans notre programme ?
11. Que se passe-t-il lorsqu'une transaction d'un bloc déjà miné est modifiée ?
12. Pourquoi le Proof of Work contribue-t-il à la sécurité d'une blockchain ?
13. Quels sont les principaux inconvénients du Proof of Work ?
14. Pourquoi notre implémentation ne constitue-t-elle pas encore une véritable blockchain décentralisée ?

Partie VI — Synthèse

Complétez le tableau suivant.

Notion	Définition ou explication	Exemple
Minage		
Proof of Work		
Nonce		
Difficulté		
Hash valide		
Preuve de travail		
Temps de minage		
Validation d'un bloc		
Intégrité de la blockchain		
Coût de modification		

Travail à rendre

Chaque étudiant doit remettre :

1. Les fichiers Python des exercices réalisés.
2. Le programme permettant de miner un bloc avec une difficulté configurable.
3. Le programme créant une blockchain de trois blocs minés.
4. La fonction de vérification de la blockchain.
5. Le tableau des mesures de temps de minage.
6. Le graphique représentant l'influence de la difficulté.

7. Les réponses aux questions des exercices et de compréhension.